

AI vs Real Image Classifier

Sarah S. Ahsan, Amna Zafar, Mannat Gabria , 02/11/2025

Innholdsfortegnelse

1. Beskriv problemet
 - 1.1 Omfang / Scope
 - 1.2 Metrikker
2. Data
 - 2.1 Datasett og strukturering
 - 2.2 Forbehandling
3. Modellering
 - 3.1 Implementasjon
 - 3.2 Resultater
 - 3.3 Diskusjon og forbedringsmuligheter
4. Deployment
 - 4.1 Webapplikasjon
 - 4.2 Videre arbeid
5. Arbeidsinnsats og tidsbruk
6. Referanser

1: BESKRIV PROBLEMET

1.1 OMFANG / SCOPE

AI - genererte bilder har blitt svært realistiske, og det blir stadig vanskeligere å skille AI - bilder fra ekte bilder. Dette skaper utfordringer spesielt innenfor tillit på nette og digital sikkerhet.

I prosjektet lager vi en modell som klassifiserer bilder som enten AI eller ekte. Vi bruker transfer learning med ResNet18 og utvikler en enkel webapplikasjon gjennom streamlit.

Det å manuelt skille bilder er krevende og ofte blir feilaktig, derfor er maskinlæring egnet til oppgaven. Målet er å bygge en maskinlæring løsning som viser god prediksjonsytelse.

1.2 METRIKKER

I prosjektet bruker vi standardklassifikasjonsmetrikker for å evaluere modellen: accuracy, precision, recall og F1 - score.

Accuracy gir en samlet vurdering av hvor mange bilder som klassifiseres riktig, mens precision, recall og F1 score brukes fordi begge klassene er viktige og feilklassifisering i begge retninger vil gi konsekvenser.

Målet er å oppnå minst 70% accuracy som første fungerende versjon av systemet, samt en balansert F1 score mellom klassene.

2: DATA

2.1 Datasett og strukturering

Datasettet vi bruker i prosjektet består av en blanding av ekte bilder og AI-genererte bilder. Formålet er å trene modellen slik at den kan skille mellom kategoriene.

Datasettet er hentet fra Kaggle og inneholder store mengder bilder fordelt på mappene real og ai_images. Bildene er i ulike stiler og kvaliteter, noe som gir modellen variasjon og gjør det mulig å klassifisere nye opplastede bilder.

Vi starter med å restrukturere datasettet ettersom bildene lå i blandede mapper. Organiseringen blir gjort ved bruk av PyTorch sin ImageFolder-funksjon i et egendefinert Python-skript kalt prepare_data.py.

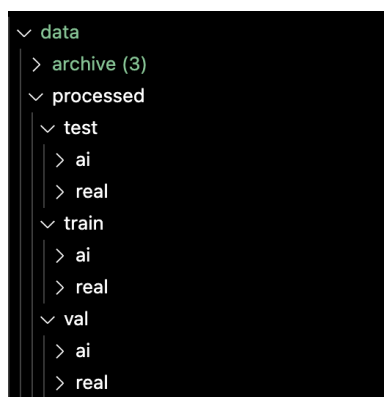
prepare_data.py automatiserer prosessen med å flytte bildene inn i riktige mapper (train/, val/, test/) basert på forhåndsdefinerte prosentandeler:

- 70% treningsdata
- 15% valideringsdata
- 15% testdata

Skriptet leser inn bildefilene fra mappene real/ og ai_image/, blander dem tilfeldig ved hjelp av random.shuffle(), og kopierer deretter filene til nye undermapper i data/processed/.

Funksjonen split_and_copy() filtrerer ut gyldige bildefiler (.jpg, .png, .jpeg), oppretter automatisk nødvendige mapper med os.makedirs(), og viser fremdriften i terminalen via tqdm-biblioteket. Automatiseringen gjør det enklere å holde datasettet ryddig, strukturert og reproducerbart for trening og testing.

Den standardiserte mappestrukturen er delt inn i:



2.2 Forbehandling

Bildene blir transformert ved hjelp av torchvision.transforms. Følgende steg er gjort:

- Resizing: bildene ble skalert til 224×224 piksler.
- Normalisering: basert på gjennomsnitt og standardavvik fra ImageNet-datasettet. Dette gjør pikselverdiene konsistente.
- Konvertering til tensor: bildene ble konvertert fra NumPy-arrays til PyTorch-tensorer.

På grunn av stort treningsdata og begrenset maskinvare benytter vi kun 20% av treningsbildene for å verifisere at modellen fungerte. Dette reduserer treningstiden samtidig som modellen oppnår gode resultater.

Denne strukturen sikrer at datasettet er:

- ryddig og strukturert,
- kompatibelt med PyTorch,

- delt mellom trening, validering og testing.

Alt dette gir et solid fundament for effektiv læring.

3: MODELLERING

3.1 Implementasjon

Hele prosjektet er strukturert i to hoveddeler:

- src/ for trening og evaluering
- webapp/ for frontend-applikasjonen via Streamlit

Til klassifikasjonen brukte vi transfer learning med ResNet18, en forhåndstrengt konvolusjonsmodell fra ImageNet.

Treningen ble implementert i train.py, med følgende parametere:

- Batch size: 32
- Learning rate: 0.0001
- Optimizer: Adam
- Loss-funksjon: CrossEntropyLoss
- Epochs: 3 i første runde, deretter økt til 6

Både trenings- og valideringsnøyaktighet ble målt etter hver epoch. Dersom valideringsnøyaktigheten økte, ble modellen lagret som model.pth. Vi beholdt den beste versjonen fremfor kun den siste som ble trent.

3.2 Resultater

Modellen blir trent i 6 epochs på 40% av treningssettet for å redusere treningstid.

Både trenings- og valideringsnøyaktighet økte i begynnelsen, men flater ut etter 3. epoch. Dette indikerer at modellen har nådd et naturlig metningspunkt uten overfitting.

```
Epoch 2: Loss= 5.0371, Train Acc = 0.5781904761904761, Val Acc= 0.6792
Lagret ny beste modell med val_acc = 0.6792
Epoch 3/6: 100%|██████████| 1313/1313 [16:44<00:00, 1.31it/s]
Epoch 3: Loss= 5.0258, Train Acc = 0.586047619047619, Val Acc= 0.6910
Lagret ny beste modell med val_acc = 0.6910
Epoch 4/6: 100%|██████████| 1313/1313 [15:44<00:00, 1.39it/s]
Epoch 4: Loss= 5.0159, Train Acc = 0.5951428571428572, Val Acc= 0.6957
Lagret ny beste modell med val_acc = 0.6957
Epoch 5/6: 100%|██████████| 1313/1313 [16:03<00:00, 1.36it/s]
Epoch 5: Loss= 5.0155, Train Acc = 0.5932619047619048, Val Acc= 0.6914
Epoch 6/6: 100%|██████████| 1313/1313 [16:13<00:00, 1.35it/s]
Epoch 6: Loss= 5.0155, Train Acc = 0.5940238095238095, Val Acc= 0.7016
Lagret ny beste modell med val_acc = 0.7016
Trening ferdig!
```

Testresultatet viser en accuracy på 69,58%. Klassifikasjonsrapporten viser balanserte resultater med F1-score på rundt 0,69 – 0,70 for begge klasser.

```
Lagret ny beste modell med val_acc = 0.7016
Trening ferdig!
🎯 Test Accuracy: 69.58%

🧩 Classification Report:

```

	precision	recall	f1-score	support
ai	0.69	0.71	0.70	15000
real	0.70	0.69	0.69	15000
accuracy			0.70	30000
macro avg	0.70	0.70	0.70	30000
weighted avg	0.70	0.70	0.70	30000

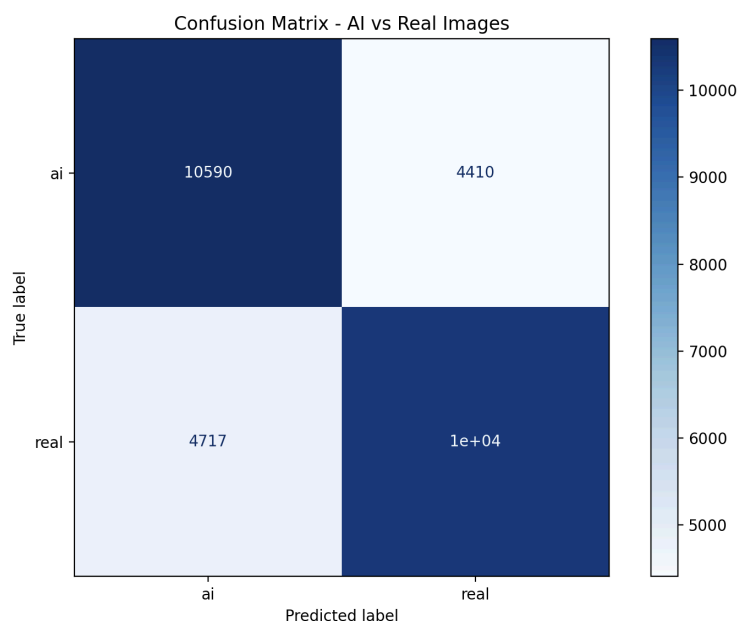
Confusion-matrisen viser at modellen klassifiserte de fleste bildene korrekt, men har en tendens til å feiltolke enkelte ekte bilder som AI.

Modellen klassifiserer mest korrekt:

- 10 590 AI bilder ble riktig identifisert som AI.
- 10 000 ekte bilder ble riktig identifisert som ekte.

Noen feil forekommer:

- 4 410 AI bilder ble feilklassifisert som ekte.
- 4 717 ekte bilder ble feilklassifisert som AI.



3.3 Diskusjon og forbedringsmuligheter

Modellen oppnådde rundt 70 % nøyaktighet og viste stabil læring uten overfitting. Feilprediksjonene oppstod hovedsakelig på ekte bilder med filtre og unaturlige trekk. Dette gjorde det vanskelig for modellen å skille dem fra AI-genererte bilder.

Med mer treningsdata og en dypere modell som ResNet50, ville ytelsen trolig forbedres.

Mer data gir bedre generalisering, og et dypere nettverk kan fange mer komplekse mønstre — men dette er også mer tidkrevende.

`predict.py` brukes til å teste modellen på nye bilder og fungerer som koblingen mellom treningsdelen og webapplikasjonene.

Skriptet oppdager automatisk riktig maskinvare (GPU, MPS eller CPU) og laster den trente ResNet18-modellen fra `model.pth`.

Bilder forbehandles med de samme transformasjonene som under trening (224×224-piksler, normalisering, konvertering til tensor) før prediksjonen utføres.

Resultatet returneres som både klasse («ai» eller «real») og sannsynlighetsverdi. Dette gjør [predict.py](#) fleksibel og enkel å bruke, samtidig som den sikrer konsistente resultater mellom treningsmiljøet og webapplikasjonen.

4. DEPLOYMENT

4.1 Webapplikasjon

Webapplikasjonen våres er utviklet i Streamlit for å teste modellen i praksis. Brukeren laster opp et bilde, og appen sender bildet til [predict.py](#), som forbehandler bildet. Deretter brukes den trente ResNet18 - modellen til å gi en prediksjon.

Appen viser om bildet er ekte eller AI-generert, sannsynligheten vises på siden. En demonstrasjon av webapplikasjonen finnes i den tilhørende videoen. (Se gjennomgang av løsningen)

4.2 Videre arbeid

Vi har planer om å trene modellen videre med flere epochs og større datamengde for å forbedre nøyaktigheten.

Videre kan en dypere modell som ResNet50 brukes for å se om den gir bedre resultater.

Et annet fokusområde er å optimalisere webapplikasjonen for bedre hastighet og brukervennlighet.

5. Arbeidsinnsats og tidsbruk

Dag	Aktivitet	Beskrivelse
1-2	Planlegging og idéutvikling	Valg av prosjektidé, oppsett av GitHub-repositorium og fordeling av oppgaver.
3-4	Datainnsamling og forbehandling	Henting av datasett fra Kaggle, organisering med <code>prepare_data.py</code> og deling i treningstestsett og valideringstestsett.
5-7	Modelltrening (<code>train.py</code>)	Implementering av ResNet18 med transfer learning, testing av hyperparametre og evaluering av treningsresultater.
8-9	Prediksjonsskript (<code>predict.py</code>)	Utvikling av skript for klassifisering av nye bilder og evaluering av modellens generaliseringsevne.
10-12	Webapplikasjon (<code>app.py</code>)	Implementering av brukergrensesnitt i Streamlit for opplasting og klassifisering av bilder. Integrasjon av modellen og testing lokalt.
13	Rapportskriving	Samling av teori, metode, resultater og diskusjon i rapportmalen.
14	Kvalitetssikring	Gjennomgang, korrektur og opprydding av kode, rapport og dokumentasjon før innlevering.

Alle gruppemedlemmene bidrar i alle deler av prosessen. Arbeidsmengden tilsvarer omtrent én uke fulltidsarbeid per medlem, fordelt over ca to uker.

6. Referanser

- Kaggle. (2025). AI vs Real Images Dataset. Hentet fra <https://www.kaggle.com/> (16.10.2025)
- OpenAI Assistant. (2025) . Veiledning brukt for strukturering og forklaring.